CSS Typography, Sizes & Units, Custom Fonts

Making text look great on the web

About These Notes

These notes cover three closely related topics — how to style text, how to measure things in CSS, and how to use custom fonts.

Beginner: Focus on understanding the properties and when to use each unit. Try every example in VSCode.

Intermediate: Pay attention to the relative units (em, rem, %, viewport), custom properties, and @font-face loading strategies.

Tools Used in This Module

VSCode + Live Server — write your CSS and HTML locally. Install Live Server for instant preview.

Google Fonts (fonts.google.com) — free custom fonts via a single <link> tag. No account needed.

Create a folder with style.css and index.html linked together. All examples in these notes go in that file.

01 CSS Typography

CSS Typography

Typography in CSS controls how text looks — the font family, weight, style, size, spacing between letters and lines, and alignment. These properties are among the most frequently used in any stylesheet.

1.1 font-family

The `font-family` property sets the typeface. Always provide a `font stack` — a comma-separated list of fonts from most preferred to least. If the first font is unavailable, the browser tries the next one.

```
style.css
/* Font stack: preferred first, generic last */
body {
  font-family: 'Inter', 'Segoe UI', Arial, sans-serif;
}

h1 {
  font-family: 'Playfair Display', Georgia, serif;
}

code {
  font-family: 'Fira Code', 'Courier New', monospace;
}
```

Beginner: Generic font families

Always end your font stack with a generic family name — this is the browser's ultimate fallback:

sans-serif — fonts without serifs (clean, modern): Arial, Helvetica

serif — fonts with serifs (traditional, editorial): Times New Roman, Georgia

monospace — equal-width characters (code): Courier New, Consolas

cursive — handwriting-style fonts

fantasy — decorative fonts

Intermediate: System font stack

A system font stack uses the OS's native UI font — fast, no download needed:

font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;

This renders SF Pro on Apple devices, Segoe UI on Windows, and Roboto on Android.

1.2 font-size

The `font-size` property sets the size of the text. There are many units — pixel-based, relative, and viewport-based. We cover all units thoroughly in Section 2.

```
style.css
body {
  font-size: 16px;        /* Base size — browsers default to 16px */
}

h1 { font-size: 2.5rem; }    /* 2.5 × root size = 40px */
h2 { font-size: 2rem; }      /* 2 × root size = 32px */
h3 { font-size: 1.5rem; }    /* 1.5 × root size = 24px */
```

```
p  { font-size: 1rem; }      /* Equal to root = 16px */
small { font-size: 0.875rem; } /* 14px */
```

1.3 font-weight

Controls how bold or light the text appears. Values can be keywords or numbers from 100 to 900. Not all fonts support every weight — only weights the font includes will render correctly.

```
style.css
p         { font-weight: 400; }   /* Normal / Regular */
strong    { font-weight: 700; }   /* Bold */
.light    { font-weight: 300; }   /* Light */
.semibold { font-weight: 600; }   /* Semi-bold */
.black    { font-weight: 900; }   /* Heaviest */
```

Value	Keyword equivalent	Appearance
100	—	Thin
200	—	Extra Light
300	light	Light
400	normal	Regular (default)
500	—	Medium
600	—	Semi-Bold
700	bold	Bold
800	—	Extra Bold
900	—	Black / Heavy

1.4 font-style

The `font-style` property makes text italic or oblique.

```
style.css
em     { font-style: italic; }    /* True italic (separate font file) */
.intro { font-style: oblique; }   /* Slanted version of the regular font
*/
cite   { font-style: italic; }
.reset { font-style: normal; }    /* Removes italic */
```

> ⚡ **Intermediate: italic vs oblique**
> italic uses a specifically designed italic variant of the font (different letterforms).

oblique simply slants the regular font — used when no italic variant exists.
For most web fonts, italic is the correct choice.

1.5 text-transform

Changes the capitalisation of text without editing the HTML.

```
style.css
.heading    { text-transform: uppercase; }   /* ALL CAPS */
.byline     { text-transform: capitalize; }  /* First Letter Of Each Word
*/
.logo       { text-transform: lowercase; }   /* all lowercase */
.disclaimer { text-transform: none; }        /* As written in HTML */
```

1.6 text-decoration

Controls underlines, overlines, and strikethroughs on text.

```
style.css
a              { text-decoration: underline; }
a:hover        { text-decoration: none; }      /* Remove on hover */
.strikethrough { text-decoration: line-through; }
.overline      { text-decoration: overline; }

/* Shorthand: line style colour thickness */
a {
  text-decoration: underline dotted #007ACC 2px;
}
```

1.7 text-align

Aligns text horizontally within its container.

```
style.css
p           { text-align: left; }     /* Default for LTR languages */
.center     { text-align: center; }
.right      { text-align: right; }
.justified  { text-align: justify; }  /* Stretched to fill the line */
```

Avoid justify on narrow columns

text-align: justify can create awkward gaps between words — especially on mobile or in narrow columns. Use it only for wide, multi-column print-like layouts.

1.8 line-height

Sets the height of each line of text — in effect, the vertical spacing between lines. A unitless value is recommended because it scales proportionally with the font size.

```
style.css
/* Unitless — scales with font size (RECOMMENDED) */
p {
  font-size: 1rem;
  line-height: 1.6;       /* 1.6 × 16px = 25.6px line height */
}

/* Other valid values */
h1 { line-height: 1.2; }  /* Tighter for headings */
p  { line-height: 24px; } /* Fixed — not recommended */
p  { line-height: 150%; } /* Percentage — works but less predictable */
```

Recommended line-height values

Body text: 1.5 – 1.8 (comfortable reading)
Headings: 1.1 – 1.3 (tighter — looks designed)
Buttons/labels: 1.0 – 1.2
WCAG accessibility guidelines recommend a minimum of 1.5 for body text.

1.9 letter-spacing and word-spacing

These control the space between individual characters and between words.

```
style.css
.heading  { letter-spacing: 0.05em; }   /* Slightly wider — common for
headings */
.caps     { letter-spacing: 0.1em; }    /* Good for uppercase labels */
.tight    { letter-spacing: -0.02em; }  /* Tighter — use sparingly */

.spaced   { word-spacing: 0.2em; }      /* More space between words */
```

1.10 text-indent and text-overflow

```
style.css
/* Indent the first line of a paragraph */
p {
  text-indent: 2em;
}
```

```
/* Truncate overflowing text with an ellipsis */
.truncate {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;  /* Shows '...' when text is cut off */
  width: 200px;
}
```

1.11 font Shorthand

The `font` shorthand combines multiple font properties into one line. The order matters.

style.css

```
/* font: [style] [variant] [weight] size/line-height font-family */
p {
  font: italic 400 1rem/1.6 'Inter', sans-serif;
}

h1 {
  font: bold 2.5rem/1.2 'Playfair Display', serif;
}
```

font shorthand resets other properties

Using the font shorthand resets any font sub-properties not explicitly listed back to their defaults.

For example: font: bold 1rem/1.6 Arial; will reset font-style to normal even if you set it earlier.

This is a common source of bugs — prefer individual properties unless you intentionally want the reset.

1.12 Typography Quick Reference

Property	Example Value	What It Controls
`font-family`	'Inter', Arial, sans-serif	Typeface / font name
`font-size`	1rem / 18px	Text size
`font-weight`	400 / bold / 700	Boldness
`font-style`	italic / normal	Italic slant
`text-transform`	uppercase / capitalize	Capitalisation
`text-decoration`	underline / none	Underline, strikethrough
`text-align`	left / center / justify	Horizontal alignment
`line-height`	1.6 / 1.4	Vertical line spacing

Property	Example Value	What It Controls
`letter-spacing`	0.05em / -0.02em	Space between characters
`word-spacing`	0.2em	Space between words
`text-indent`	2em	First-line indentation
`text-overflow`	ellipsis	Truncation with ...

Practice Task — Typography (VSCode)

Create a file called typography.html with a linked style.css.
Build a simple article page with a heading, byline, and three paragraphs.
Style it using:

- A Google Font for the heading (Playfair Display or Merriweather)
- A sans-serif stack for body text
- font-size: 1.1rem and line-height: 1.7 on the <p> elements
- text-transform: uppercase and letter-spacing: 0.1em on the byline
- A .truncate class that cuts long text with an ellipsis

Open with Live Server and adjust values until the text feels comfortable to read.

02 CSS Sizes & Units

CSS Sizes & Units

In CSS, wherever you set a measurement — font size, padding, width, margin, border — you choose a unit. Picking the right unit is one of the most important skills in CSS. The wrong unit makes layouts break on different screen sizes; the right unit makes them adapt automatically.

2.1 Absolute Units

Absolute units have a fixed size that does not change regardless of the screen, parent element, or user's settings.

Unit	Full Name	Equivalent	When to Use
`px`	Pixel	1px = 1 device pixel (approx)	Borders, shadows, precise sizing
`pt`	Point	1pt = 1/72 inch	Print stylesheets only

Unit	Full Name	Equivalent	When to Use
`pc`	Pica	1pc = 12pt	Print only
`cm`	Centimetre	1cm = 37.8px	Print only
`mm`	Millimetre	1mm = 3.78px	Print only
`in`	Inch	1in = 96px	Print only

Beginner: When to use px

px is your starting point and is fine for borders, shadows, and small fixed details.

However, using px for font-size and spacing can cause problems: if a user increases their browser's default font size (for accessibility), px values will NOT scale — rem values will.

Rule of thumb: use px for visual decoration, use rem/em for text and spacing.

2.2 Relative Units — em and rem

These are the two most important relative units for typography and spacing.

em — relative to the parent's font-size

1em equals the font-size of the element's direct parent. If the parent has `font-size: 20px`, then `1em = 20px` for that element's children.

style.css

```css
/* em compounds through nested elements */
.parent {
  font-size: 20px;
}
.parent .child {
  font-size: 1.5em;    /* 1.5 × 20px = 30px */
  padding: 1em;        /* 1 × 30px = 30px  (uses OWN font-size) */
}
.parent .child .grandchild {
  font-size: 0.8em;    /* 0.8 × 30px = 24px — compounds! */
}
```

em compounds — be careful with nesting

Each nested element multiplies the em from its parent. In deeply nested HTML this leads to unpredictable font sizes.

List item at 0.9em inside a list at 0.9em = 0.81em of the root

List item inside that = 0.729em — and so on.

This is why rem was introduced.

rem — relative to the root font-size

1rem always equals the font-size of the `<html>` element (the 'root'). It does NOT compound through nesting. This makes it far more predictable than em for font sizes.

```
style.css
/* Set the root — 1rem = 16px everywhere */
html {
  font-size: 16px;   /* Browser default is also 16px */
}

h1     { font-size: 2.5rem; }   /* 2.5 × 16 = 40px — always */
h2     { font-size: 2rem; }     /* 32px — always */
p      { font-size: 1rem; }     /* 16px — always */
small  { font-size: 0.875rem; } /* 14px — always */

/* Tip: set html font-size to 62.5% → 1rem = 10px (easier maths) */
html   { font-size: 62.5%; }
p      { font-size: 1.6rem; }   /* 1.6 × 10 = 16px */
```

⚡ Intermediate: rem and user accessibility

If you set the root in px (html { font-size: 16px; }) you override the browser's default and break user font scaling.

Better approach: set html { font-size: 100%; } (or don't set it at all) so the browser default (16px) applies, and rem scales correctly when the user changes their font size in browser settings.

Then all your rem values will scale with the user's preference — this is an accessibility win.

2.3 Relative Units — Percentage (%)

Percentages are always relative to something — what that 'something' is depends on the property.

```
style.css
.container {
  width: 80%;         /* 80% of the parent element's width */
}

p {
  font-size: 120%;    /* 120% of the parent's font-size */
}

.box {
  padding: 5%;        /* 5% of the CONTAINING BLOCK'S WIDTH (not height) */
  margin: 0 auto;     /* auto margins centre a block element */
}
```

% for width is great — % for font-size is tricky

Percentage widths are excellent for fluid, responsive layouts.

Percentage font sizes compound just like em — each nested element multiplies from the parent. Prefer rem for font sizes.

2.4 Viewport Units

Viewport units are relative to the browser window (viewport) size — extremely useful for responsive layouts.

Unit	Relative To	Example Use
`vw`	1% of the viewport width	Fluid text, full-width sections
`vh`	1% of the viewport height	Full-screen hero sections
`vmin`	1% of the smaller viewport dimension	Square elements that fit any screen
`vmax`	1% of the larger viewport dimension	Large background typography
`svh`	1% of the small viewport height (mobile)	Mobile hero sections (avoids toolbar shift)
`dvh`	1% of the dynamic viewport height	Mobile — adjusts as browser chrome hides

style.css

```css
/* Full-screen hero section */
.hero {
  height: 100vh;
  width: 100vw;
}

/* Fluid headline — scales with the screen */
.headline {
  font-size: clamp(1.5rem, 5vw, 4rem);
  /* min: 1.5rem  preferred: 5vw  max: 4rem */
}
```

⚡ Intermediate: clamp() for fluid typography

clamp(min, preferred, max) is the modern way to create fluid type that grows with the viewport but stays within safe bounds:

font-size: clamp(1rem, 2.5vw, 2rem);

→ minimum 1rem, grows at 2.5vw, capped at 2rem

This replaces multiple media queries for font sizing. Supported in all modern browsers.

2.5 ch and ex Units

Unit	Meaning	Best Use
`ch`	Width of the '0' character in the current font	Setting readable line lengths — 60–75ch is ideal for body text
`ex`	Height of the lowercase 'x' in the current font	Vertical alignment of inline elements

```
style.css
/* Optimal reading width: 60–75 characters */
article p {
  max-width: 70ch;
}
```

max-width: 70ch is one of the best typography tricks

Setting a maximum width in ch units on body text ensures lines stay between 60–75 characters wide — the typographic sweet spot for comfortable reading on any screen size.

2.6 CSS Custom Properties (Variables) for Spacing

CSS custom properties (also called CSS variables) let you define values once and reuse them throughout your stylesheet. They are especially powerful for consistent spacing and typography scales.

```
style.css
:root {
  /* Typography scale */
  --text-xs:   0.75rem;   /*  12px */
  --text-sm:   0.875rem;  /*  14px */
  --text-base: 1rem;      /*  16px */
  --text-lg:   1.125rem;  /*  18px */
  --text-xl:   1.25rem;   /*  20px */
  --text-2xl:  1.5rem;    /*  24px */
  --text-3xl:  1.875rem;  /*  30px */
  --text-4xl:  2.25rem;   /*  36px */

  /* Spacing scale */
  --space-1:  0.25rem;   /*  4px  */
  --space-2:  0.5rem;    /*  8px  */
  --space-4:  1rem;      /*  16px */
  --space-8:  2rem;      /*  32px */
  --space-16: 4rem;      /*  64px */
}

h1 { font-size: var(--text-4xl); }
p  { font-size: var(--text-base); margin-bottom: var(--space-4); }
```

⚡ Intermediate: Why a design system uses rem-based scales

Using rem-based custom properties creates a consistent, scalable system.

If you change the root font size (e.g. for a specific screen or user preference), every value in your system scales automatically — no need to update individual values.

Frameworks like Tailwind CSS, Material UI, and Chakra UI are all built on this principle.

2.7 Units Quick Reference

Unit	Type	Relative To	Best Used For
`px`	Absolute	Screen pixel	Borders, shadows, icons
`rem`	Relative	Root (<html>) font-size	Font sizes, spacing — preferred
`em`	Relative	Parent element font-size	Padding/margin relative to text size
`%`	Relative	Parent (varies by property)	Widths, fluid layouts
`vw`	Viewport	Viewport width	Fluid widths, hero sections
`vh`	Viewport	Viewport height	Full-screen sections
`ch`	Relative	Width of '0' character	Readable line-length limits
`vmin`	Viewport	Smaller viewport dimension	Square responsive elements
`dvh`	Viewport	Dynamic viewport height	Mobile hero sections

□ Practice Task — Units (VSCode)

In style.css, add a :root block with a full spacing and type scale using custom properties.

Then create a card component and apply sizes exclusively using the variables:

- font-size: var(--text-xl) on the card title
- padding: var(--space-4) on the card body
- max-width: 70ch on any paragraph
- font-size: clamp(1rem, 3vw, 2rem) on the page heading

Resize your browser window and watch how clamp() and vw keep the heading fluid.

03 Custom Fonts

Custom Fonts

Web-safe fonts (Arial, Times New Roman, Georgia) are available on every device but offer limited creative choice. Custom fonts let you use any typeface — the font file is downloaded by the browser when your page loads. There are two main methods: using a font service like Google Fonts, or self-hosting with @font-face.

3.1 Google Fonts — The Quickest Method

Google Fonts provides hundreds of free, open-source fonts via a CDN. No account or download needed — just a <link> tag in your HTML and a font-family declaration in your CSS.

Step-by-step: Adding a Google Font in VSCode

1. Go to fonts.google.com in your browser.
2. Search for a font (e.g. 'Inter' for body text, 'Playfair Display' for headings).
3. Click 'Get Font' then 'Get embed code'.
4. Copy the <link> tags shown under 'Embed code in the <head> of your html'.
5. Paste them into your HTML <head> BEFORE your own <link rel='stylesheet'>.
6. Copy the font-family CSS rule and add it to your stylesheet.

index.html

```
<!-- In HTML <head> — paste Google Fonts link here -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;700&family=
Playfair+Display:wght@700&display=swap" rel="stylesheet">
```

style.css

```
/* In style.css — use the font by name */
body {
  font-family: 'Inter', sans-serif;
  font-weight: 400;
}

h1, h2, h3 {
  font-family: 'Playfair Display', serif;
  font-weight: 700;
}
```

Load only the weights you need

Each font weight is a separate file download. Loading Inter:wght@100;200;300;400;500;600;700;800;900 downloads 9 files.

Only request the weights you will actually use. For most projects, 400 (regular) and 700 (bold) are enough.

In the Google Fonts URL above: wght@400;500;700 loads only those three weights.

> ⚡ **Intermediate: display=swap explained**
>
> The display=swap parameter in the Google Fonts URL tells the browser to use a fallback font immediately and swap to the custom font once it downloads — preventing a blank flash of invisible text (FOIT).
>
> This is a performance and user experience best practice. Always include it.
>
> Alternative values: display=fallback (shorter swap period), display=optional (may not swap at all on slow connections).

3.2 Variable Fonts

Variable fonts pack multiple weights, widths, and styles into a single font file. Instead of separate files for 400, 500, 700, one variable font file handles the entire range — faster to load and more flexible.

```
style.css
/* Google Fonts variable font — note the range syntax 100..900 */
/* URL: fonts.googleapis.com/css2?family=Inter:wght@100..900&display=swap
*/

body  { font-weight: 400; }
h1    { font-weight: 750; }   /* Any value from 100-900, not just
multiples of 100 */
.lead { font-weight: 450; }   /* Not possible with static fonts */
```

> ⚡ **Intermediate: Checking for variable font support**
>
> On Google Fonts, look for 'Variable' in the font details. Inter, Roboto Flex, and Source Sans 3 are popular variable fonts.
>
> Variable fonts use the font-variation-settings property for custom axes beyond weight:
>
> font-variation-settings: 'wght' 650, 'wdth' 75;
>
> Check support and available axes at v-fonts.com.

3.3 Self-Hosting with @font-face

When you host your own font files, use the `@font-face` rule to register them. This gives you full control over loading behaviour, file formats, and fallbacks — and keeps your page independent of third-party services.

> □ **Getting font files for self-hosting**
>
> 1. Download a font from Google Fonts: on the font page, click 'Download family' to get TTF files.

2. For better performance, convert TTF to WOFF2 using the free tool: transfonter.org or fontsquirrel.com/tools/webfont-generator.
3. Create a 'fonts/' folder inside your project and place the files there.
4. Reference them with @font-face in your CSS.

```
style.css
/* Minimum @font-face setup */
@font-face {
  font-family: 'Inter';
  src: url('fonts/inter-regular.woff2') format('woff2'),
       url('fonts/inter-regular.woff')  format('woff');
  font-weight: 400;
  font-style: normal;
  font-display: swap;
}

@font-face {
  font-family: 'Inter';
  src: url('fonts/inter-bold.woff2') format('woff2'),
       url('fonts/inter-bold.woff')  format('woff');
  font-weight: 700;
  font-style: normal;
  font-display: swap;
}

/* Now use it like any other font */
body {
  font-family: 'Inter', sans-serif;
}
```

@font-face descriptor	Purpose	Recommended value
`font-family`	The name you give this font — use it in font-family declarations	Your chosen name in quotes, e.g. 'Inter'
`src`	Path to the font file — list WOFF2 first, WOFF second	url() format() — always WOFF2 first
`font-weight`	Which weight this file represents	Match the numeric value: 400, 700 etc.
`font-style`	Whether this file is the normal or italic variant	normal or italic
`font-display`	How the browser handles loading — controls FOUT/FOIT	swap — show fallback, swap when ready

font-display values explained

font-display value	Behaviour
`auto`	Browser decides — usually same as block
`block`	Invisible text for up to 3 seconds, then swap. Causes FOIT.
`swap`	Show fallback immediately, swap when font loads. Causes FOUT but best for readability.
`fallback`	Invisible for 100ms, fallback for ~3s, then no more swapping.
`optional`	100ms invisible, then browser may skip loading if connection is slow. Best performance.

Use font-display: swap for most projects

swap gives the best user experience in most cases — text is always readable, even before the custom font loads.

Use optional for decorative fonts that are purely aesthetic — if they don't load, no content is lost.

3.4 WOFF2 vs WOFF vs TTF

Format	Extension	Browser Support	Recommended?
`WOFF2`	.woff2	All modern browsers — Chrome, Firefox, Safari, Edge	✓Yes — always use as primary
`WOFF`	.woff	Slightly older browsers — IE9+	✓As fallback after WOFF2
`TTF`	.ttf	Wide support but no compression	Only if WOFF2/WOFF unavailable
`EOT`	.eot	IE only (obsolete)	✗Do not use
`SVG`	.svg	Old iOS Safari only (obsolete)	✗Do not use

⚡ Intermediate: WOFF2 compression

WOFF2 uses the Brotli compression algorithm — fonts are typically 30% smaller than WOFF and up to 60% smaller than TTF.

For a typical Inter Regular file: TTF ≈ 290KB, WOFF ≈ 140KB, WOFF2 ≈ 110KB.

Smaller files = faster page loads = better Core Web Vitals scores.

3.5 Preloading Fonts for Performance

Font files are discovered late in the browser's loading process — after the CSS is parsed. Use the `<link rel='preload'>` tag to tell the browser to download the font early.

```
index.html
<!-- In HTML <head> — tell browser to download font ASAP -->
<link rel="preload"
      href="fonts/inter-regular.woff2"
      as="font"
      type="font/woff2"
      crossorigin>

<!-- Then @font-face registers it in CSS as usual -->
```

> ⚡ **Intermediate: Only preload critical fonts**
>
> Preloading too many fonts can actually slow your page down — the browser prioritises preloaded resources over other critical assets.
>
> Preload only: the font used for above-the-fold body text and the primary heading font.
>
> Do not preload: icon fonts, decorative fonts used only in footers or modals.

3.6 Complete Example — Typography System

Putting Sections 1, 2, and 3 together into a complete, production-ready CSS file:

```
style.css
/* ── FONT IMPORT (Google Fonts) ──────────────────────── */
/* In <head>: <link href='https://fonts.googleapis.com/
   css2?family=Inter:wght@400;500;700&family=Playfair+
   Display:wght@700&display=swap' rel='stylesheet'> */

/* ── ROOT: TOKENS ────────────────────────────────────── */
:root {
  --font-body:    'Inter', sans-serif;
  --font-heading: 'Playfair Display', serif;

  --text-sm:   0.875rem;
  --text-base: 1rem;
  --text-lg:   1.125rem;
  --text-xl:   1.25rem;
  --text-2xl:  1.5rem;
  --text-4xl:  2.25rem;

  --leading-tight:  1.2;
  --leading-normal: 1.6;
  --leading-loose:  1.8;

  --space-2: 0.5rem;
  --space-4: 1rem;
  --space-8: 2rem;
}

/* ── RESET + BASE ────────────────────────────────────── */
```

```
html {
  font-size: 100%;
}

body {
  font-family: var(--font-body);
  font-size: var(--text-base);
  font-weight: 400;
  line-height: var(--leading-normal);
  color: #212121;
}

/* ── HEADINGS ──────────────────────────────────────────── */
h1 {
  font-family: var(--font-heading);
  font-size: clamp(var(--text-2xl), 5vw, var(--text-4xl));
  font-weight: 700;
  line-height: var(--leading-tight);
  letter-spacing: -0.02em;
  margin-bottom: var(--space-4);
}

h2 {
  font-family: var(--font-heading);
  font-size: var(--text-2xl);
  font-weight: 700;
  line-height: var(--leading-tight);
  margin-bottom: var(--space-2);
}

/* ── BODY TEXT ─────────────────────────────────────────── */
p {
  font-size: var(--text-base);
  line-height: var(--leading-normal);
  max-width: 70ch;
  margin-bottom: var(--space-4);
}

/* ── UTILITY CLASSES ──────────────────────────────────── */
.text-sm    { font-size: var(--text-sm); }
.text-lg    { font-size: var(--text-lg); }
.font-bold  { font-weight: 700; }
.uppercase  { text-transform: uppercase; letter-spacing: 0.1em; }
.truncate   { white-space: nowrap; overflow: hidden; text-overflow:
ellipsis; }
```

3.7 Choosing Fonts — Practical Guidance

Pairing fonts well

A reliable starting rule: one serif heading font + one sans-serif body font.

Good pairings from Google Fonts:
Playfair Display (heading) + Source Sans 3 (body)
Merriweather (heading) + Open Sans (body)
DM Serif Display (heading) + DM Sans (body) — same family, guaranteed harmony
Lora (heading) + Nunito (body)
Avoid mixing more than two typefaces on one page.

Use case	Recommended fonts	Why
`Body text`	Inter, Source Sans 3, Open Sans	High legibility at small sizes, large character support
`Editorial / blog`	Merriweather, Lora, Playfair Display	Serif rhythm improves long-form reading
`Headings`	Playfair Display, DM Serif, Raleway	Strong personality, high contrast at large sizes
`Technical / code`	Fira Code, JetBrains Mono, Source Code Pro	Ligatures, clear distinction between 0/O, l/1/I
`Minimal / modern`	Inter, Figtree, Plus Jakarta Sans	Clean, neutral, works for UI and marketing

∑ Summary & Quick Reference

Summary & Quick Reference

CSS Typography Properties

Property	Purpose
`font-family`	Typeface — always use a stack ending in a generic family
`font-size`	Text size — prefer rem for scalable, accessible sizing
`font-weight`	Boldness — 400 normal, 700 bold; any value 100–900 for variable fonts
`font-style`	Italic or normal
`text-transform`	Uppercase, lowercase, capitalize
`text-decoration`	Underline, strikethrough, overline
`text-align`	Left, center, right, justify

Property	Purpose
`line-height`	Vertical line spacing — unitless (e.g. 1.6) recommended
`letter-spacing`	Space between characters — em unit recommended
`text-overflow`	Truncation with ellipsis — needs overflow:hidden + white-space:nowrap

CSS Units Cheat Sheet

Unit	Best For	Avoid For
`px`	Borders, shadows, precise icons	Font sizes (blocks user scaling)
`rem`	Font sizes, consistent spacing	—
`em`	Padding/margin relative to text	Font sizes in nested elements
`%`	Widths, fluid container sizing	Font sizes (compounds like em)
`vw/vh`	Hero sections, full-screen layouts	Body font sizes (use clamp instead)
`ch`	Readable max-width on paragraphs	—
`dvh`	Mobile hero sections	—

Custom Fonts Cheat Sheet

Method	When to Use	Key Code
`Google Fonts <link>`	Quickest — prototypes and most projects	<link href='fonts.googleapis.com/...' rel='stylesheet'>
`@font-face self-host`	Full control, no third-party dependency	@font-face { src: url('fonts/font.woff2') format('woff2'); }
`Variable font`	One file covers all weights — faster load	Inter:wght@100..900 on Google Fonts
`<link rel=preload>`	Speed up above-the-fold font rendering	<link rel='preload' as='font' href='font.woff2' crossorigin>

Key Rules to Remember

1. Always end a font stack with a generic family: `sans-serif`, `serif`, or `monospace`.
2. Use `rem` for font sizes and spacing — it respects the user's browser font setting.

3. Use `em` for padding and margin when you want them to scale with the element's own text size.
4. Use `max-width: 70ch` on body text paragraphs for optimal reading line length.
5. Always include `font-display: swap` in `@font-face` to prevent invisible text while loading.
6. Load only the font weights you actually use — each weight is a separate download.
7. Use `clamp(min, preferred, max)` for fluid typography that scales between screen sizes.

Final Practice Task — Full Typography System (VSCode)

Create a project folder with index.html and style.css.

1. Add two Google Fonts in index.html — one serif for headings, one sans-serif for body.
2. In style.css, define a :root block with type scale and spacing variables.
3. Style body, h1, h2, p using the variables.
4. Add font-size: clamp(...) to the h1.
5. Set max-width: 70ch on all paragraphs.
6. Add a .truncate utility class.
7. Create a second version using @font-face with a downloaded WOFF2 file.
8. Compare loading in DevTools (F12 → Network tab → filter by 'Font').

Stretch goal: convert the Google Fonts link to a self-hosted @font-face and verify the font still loads correctly with Live Server.

Further Resources

Google Fonts: https://fonts.google.com
Variable fonts showcase: https://v-fonts.com
Font format converter: https://transfonter.org
Typography scale generator: https://typescale.com
MDN CSS font properties: https://developer.mozilla.org/en-US/docs/Web/CSS/font
Web.dev typography guide: https://web.dev/learn/css/typography